

Software Architecture

2026-2027

1 Project 1

1.1 Context

In a previous project, the Library Management REST-oriented backend service was developed. This application provides REST endpoints for managing:

1. Books
2. Genres
3. Authors
4. Readers
5. Lendings

1.2 Problem

The Library Management service does not adequately support:

1. Extensibility
2. Configurability
3. Testability

1.3 Goals

The goals of this project (P1) are:

1.3.1 Documenting

1. System-as-is (reverse engineering design)
2. Requirements, especially ASR
3. Architectural views and models
4. Traceability and consistency between requirements, architectural decisions, architectural views/models, and the implemented solution

1.3.2 Adoption of ADD

1. Identification and classification of requirements and architectural factors
2. ADD process

3. Tactics
4. Reference architectures
5. Patterns
6. Architectural design alternatives and rationale (visual models)

1.3.3 Development

Make the developments required so that the project supports:

1.3.3.1 Bibliographic information

1. Retrieving bibliographic information about books from different external systems, including Google Books and Open Library. External systems may expose different data models and provide different subsets of bibliographic information.
2. Selecting, at setup time, the external system(s) used to retrieve bibliographic information, including the possibility of using more than one source.
3. Supporting new bibliographic information sources with limited impact on existing functionality.

1.3.3.2 Reader notifications

1. Notifying readers of relevant lending events using different notification mechanisms, including email, SMS and webhook, individually or in combination.
2. Selecting, at setup time, the notification mechanism(s) to be used.
3. Supporting new notification mechanisms with limited impact on existing functionality.

1.3.3.3 Lending policies

1. Applying different lending policies according to the complementary lending policy specification.
2. Supporting externally configurable lending policy parameters at setup time, without requiring changes to the source code.
3. Selecting the applicable lending policy at runtime according to the defined lending policy selection rules.
4. Supporting new lending policies with limited impact on existing functionality.
5. Lending policies business rules to be defined

1.3.3.4 Automated Functional Testing

To improve automated functional testing:

1. Functional opaque-box with SUT = classes
2. Functional transparent-box with SUT = domain classes
3. Mutation tests with SUT = classes
4. Functional opaque-box with SUT = controller + service + {domain, repository, gateways}
5. Functional opaque-box with SUT = system
6. Tests provide the key evidence for evaluating the functionality

1.4 Conditions

- Number of students per group:
 - 2
 - exceptionally 3 or 1, if accepted by faculty, and implies differences in goals
- First evaluation:
 - Mandatory
 - Submission deadline: 2026-11-01
 - Mode: Individual, synchronous, face-to-face
 - Weight: 100% (or 70% if second evaluation takes places)
- Second evaluation:
 - Optional
 - Submission deadline: (as P2 submission deadline)
 - Mode: Individual, synchronous, face-to-face
 - Weight: 30% (or 0% if it does not take place)

2 Project 2

TBD